

Designing Enterprise Retrieval-Augmented Generation Systems

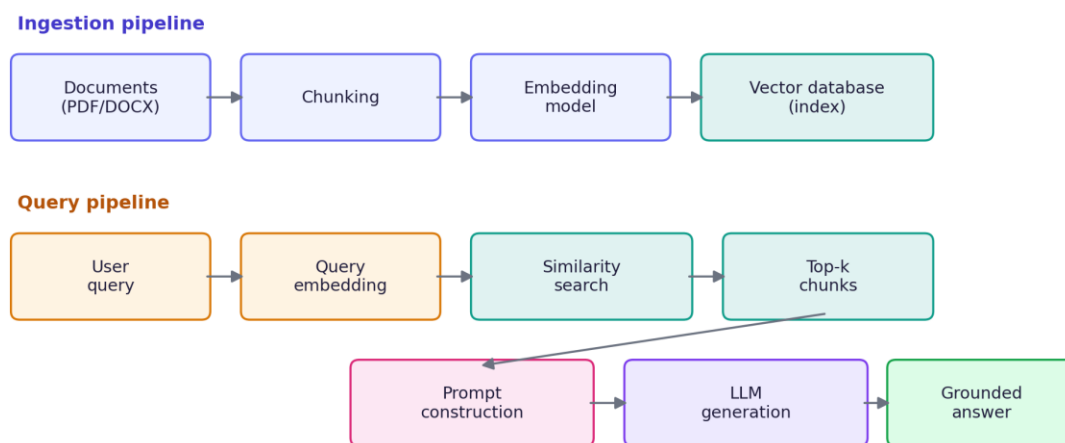
Technical Research Report

1. Introduction

Retrieval-Augmented Generation (RAG) combines a large language model (LLM) with an external knowledge source, allowing the model to answer questions using retrieved, up-to-date, and domain-specific information rather than relying solely on its static training data. This is critical for enterprise use cases, where accuracy, traceability, and freshness of information are essential and hallucinated answers carry real business and compliance risk. This report examines the core architectural components required to design a production-grade enterprise RAG system.

2. System Architecture

An enterprise RAG system is composed of two parallel pipelines: an ingestion pipeline that prepares organizational documents for retrieval, and a query pipeline that answers user questions at runtime using that indexed knowledge.



In production, this architecture is typically deployed with a decoupled ingestion service (batch or event-driven, triggered on document upload) and a stateless query service that scales independently based on user traffic.

3. Embeddings

Embeddings are dense numerical vector representations of text that capture semantic meaning, allowing similarity between pieces of text to be measured mathematically (typically via cosine similarity). Enterprise systems must choose between:

- Proprietary embedding APIs (OpenAI, Cohere) higher quality, ongoing per-token cost, external dependency
- Open-source local models (sentence-transformers, BGE, E5) — no per-call cost, full data control, requires compute

Embedding model choice directly impacts retrieval quality and must remain consistent between the ingestion and query pipelines, since embeddings from different models are not directly comparable.

4. Chunking

Documents must be split into smaller chunks before embedding, since embedding models have limited context windows and retrieval precision degrades with overly large chunks. Key strategies include:

- Fixed-size chunking with overlap (e.g. 500 tokens, 50-token overlap) — simple, consistent
- Semantic/structural chunking — splits along headings, paragraphs, or sentence boundaries for more coherent chunks
- Recursive chunking — falls back to smaller splits only where needed, preserving structure where possible

Chunk size is a trade-off: smaller chunks improve retrieval precision but risk losing surrounding context; larger chunks preserve context but dilute relevance signal and increase prompt cost.

5. Retrieval

At query time, the user's question is embedded using the same model used during ingestion, and the vector store returns the top-k most similar chunks. Enterprise systems often extend this baseline retrieval with:

- Hybrid search — combining dense vector similarity with traditional keyword (BM25) search
- Re-ranking — a secondary, more expensive model re-scores the top candidates for precision
- Metadata filtering — restricting retrieval to specific document types, dates, or access-permitted sources

6. Vector Databases

The vector database indexes embeddings for efficient approximate nearest neighbor (ANN) search at scale. Common enterprise options:

Database	Strengths	Trade-offs
Pinecone	Fully managed, low ops overhead, scales elastically	Usage-based cost, external network dependency
Weaviate / Qdrant	Self-hostable, hybrid search, open source	Requires infrastructure and operational maintenance
pgvector	Reuses existing Postgres/MySQL infrastructure	Less optimized for very large-scale ANN search
ChromaDB	Lightweight, simple local setup, good for prototypes	Less battle-tested at enterprise scale

7. Prompt Construction

Retrieved chunks are not sent to the LLM directly; they are assembled into a structured prompt that instructs the model to answer strictly using the supplied context. A well-constructed prompt typically includes: a system instruction defining scope and tone, the retrieved context clearly delimited, the user's original question, and an explicit instruction to decline or flag when the context is insufficient. Prompt structure materially affects both answer quality and hallucination rate.

8. Hallucination Reduction

Because LLMs can generate plausible but incorrect text even with relevant context supplied, enterprise RAG systems layer multiple safeguards to keep answers grounded and verifiable:

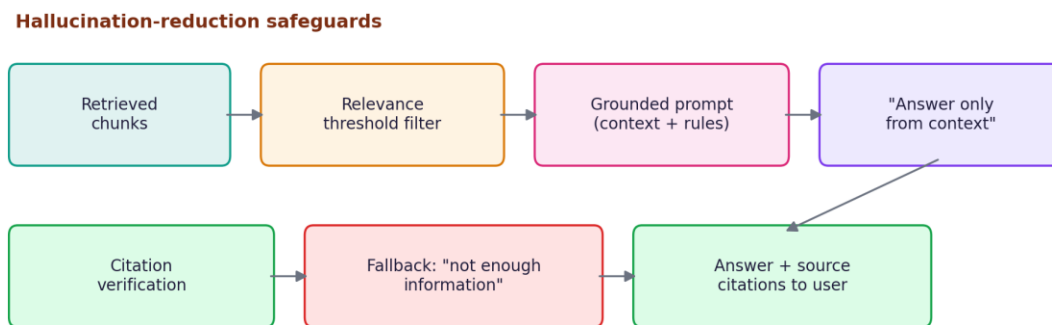


Figure 2: Hallucination-reduction pipeline

- Relevance-score thresholds to discard weakly related chunks before they reach the prompt
- Explicit prompt instructions restricting the model to the supplied context
- Citation of source chunks alongside every answer, enabling user verification
- Fallback responses (“not enough information”) when no sufficiently relevant context is retrieved

9. Future Improvements

- Agentic RAG — allowing the system to issue multiple retrieval queries and reason iteratively before answering
- Adaptive chunking — dynamically sizing chunks based on document structure and content density
- Feedback-driven re-ranking — using user thumbs-up/down signals to continuously improve retrieval ranking
- Multi-modal retrieval — extending beyond text to retrieve from tables, images, and structured data
- Fine-grained access control — enforcing per-user document permissions at retrieval time for enterprise compliance

10. Conclusion

Enterprise RAG systems require careful design across the full pipeline — from chunking and embedding strategy through retrieval, prompt construction, and hallucination safeguards. No single component determines system

quality; rather, it is the combination of well-tuned retrieval, grounded prompting, and verification mechanisms that produces a reliable, trustworthy production system.